

DSA Fundamentals 30 questions with compact answers

Use this as a worked-example booklet, then practice writing the solutions with the answers covered. Each algorithm page groups the task, starter code, follow-ups, solution, and complexity.

A practical way to prepare

1. Learn one pattern. Read its example, trace the variables, and explain why the data structure fits.
2. Cover the answer. Restate the contract, then write the implementation from a blank editor.
3. Test aloud. Use the sample, an empty or smallest input, and a boundary case such as duplicates, negatives, or an invalid k .
4. Explain the cost. Identify how many times each item is visited and which extra structures grow.
5. Revisit and mix. Reimplement later, then solve a related problem whose condition or output differs.

Karat recommends practice problems followed by more realistic practice under time pressure while explaining the approach aloud. A readable answer key supports learning; reproducing the algorithm independently checks readiness.

A suggested 40-minute practice block

Minutes	What to practice
0-5	Clarify input, output, assumptions, and one example.
5-20	Implement one foundation problem without viewing the answer.
20-35	Attempt a related variant or a harder follow-up.
35-40	Run edge cases and explain time, space, and one invariant.

Conventions used throughout

Indices are zero-based. Inputs satisfy stated type and ordering assumptions. Hash-map/set costs are expected-case. Auxiliary space excludes returned output only where explicitly noted. n counts inputs or nodes; h is tree height and w is maximum level width. Standard complexity assumes constant-cost arithmetic; very large integer results use more bit storage and arithmetic work.

Readiness check: can you explain the invariant, write the code without copying, test it, and justify its costs? If a pattern is unclear, trace a smaller input before adding a timer.

Find the pattern

Arrays and strings: 01-08 | Linked lists, stacks, queues: 09-14 | Trees: 15-18 | Scans and windows: 19-23 | Recursion: 24-27 | Extra fundamentals: 28-30.

No.	Topic	Core pattern
01	Two Sum	Complement lookup
02	Contains Duplicate	Seen set
03	Frequency counting	Count map
04	Top K Frequent Elements	Min heap
05	Anagram detection	Character counts
06	Longest unique substring	Nested scan baseline
07	String compression	Read/write pointers
08	Reverse words	Split and reverse
09	Reverse a linked list	Pointer rewiring
10	Detect a linked-list cycle	Slow/fast identity
11	Find the middle node	Slow/fast midpoint
12	Valid parentheses	LIFO matching
13	Min stack	Minimum per depth
14	Queue implementation	Two-stack FIFO
15	Tree traversal	Iterative DFS
16	Tree traversal	Queue BFS
17	Maximum tree depth	Recursive combination
18	Validate a BST	Ancestor bounds
19	Maximum sum subarray	Kadane recurrence
20	Fixed-size window	Fixed window
21	Longest unique substring	Last-seen window
22	Minimum-length subarray reaching a target	Shrink qualifying window
23	Longest substring with at most k distinct characters	Distinct-count window
24	Fibonacci	Memoized recursion
25	Recursive tree traversal	Recursive DFS
26	Backtracking basics	Subsets
27	Backtracking basics	Permutations
28	Binary search	Binary search
29	Sorted Two Sum	Two pointers
30	Climbing stairs	Two-state DP

01. Two Sum: remember the complement

ARRAYS / HASH MAP

```
def two_sum(nums, target):  
    pass  
  
print(two_sum([2, 7, 11, 15], 9))  
print(two_sum([3, 3], 6))
```

QUESTIONS

1. Return indices of two distinct elements that sum to target. Return [] if no pair exists; any valid pair is acceptable. Do not modify nums.
2. Trace the two calls. What information must you remember from earlier elements?
3. Give the time and extra-space costs. Explain why the lookup must happen before storing the current element.

COMPACT ANSWERS

1. Python solution:

```
def two_sum(nums, target):  
    seen = {}  
    for i, value in enumerate(nums):  
        needed = target - value  
        if needed in seen:  
            return [seen[needed], i]  
        seen[value] = i  
    return []
```

2. Output: [0, 1], then [0, 1]. For 2, remember index 0; at 7, the needed 2 is already present. Equal values work when they occupy different indices.
3. Expected $O(n)$ time, $O(n)$ extra space. Looking up first prevents using the current index twice (for example, a single 3 with target 6). Empty input and no match return [].

02. Contains Duplicate: track what you have seen

ARRAYS / SET

```
def contains_duplicate(nums):  
    pass  
  
print(contains_duplicate([1, 2, 3, 1]))  
print(contains_duplicate([1, 2, 3]))
```

QUESTIONS

1. Return True if any integer appears at least twice, otherwise False. Do not modify the input.
 2. Trace the calls and state when you can stop scanning.
 3. Give complexity and test empty input, one element, and a duplicate at the end.
-

COMPACT ANSWERS

1. Python solution:

```
def contains_duplicate(nums):  
    seen = set()  
    for value in nums:  
        if value in seen:  
            return True  
        seen.add(value)  
    return False
```

2. Output: True, then False. Stop immediately when the current value is already in seen. The set contains the values processed before this iteration.
3. Expected $O(n)$ time and $O(n)$ extra space in the worst input size. `[]` and a single value return False. `set(nums)` is a shorter alternative, but always constructs the complete set.

03. Frequency counting: build a histogram

ARRAYS / DICTIONARY

```
def frequencies(values):  
    pass  
  
print(frequencies(["A", "B", "A", "C", "B", "A"]))  
print(frequencies([]))
```

QUESTIONS

1. Return a dictionary mapping each hashable value to its occurrence count. Preserve first-seen key order.
2. Trace both calls. Why does `direct counts[value] += 1` fail on a previously unseen value?
3. State complexity in terms of n inputs and m distinct values. How would you select the most frequent value?

COMPACT ANSWERS

1. Python solution:

```
def frequencies(values):  
    counts = {}  
    for value in values:  
        counts[value] = counts.get(value, 0) + 1  
    return counts
```

```
{'A': 3, 'B': 2, 'C': 1}  
{}
```

2. `get(value, 0)` supplies the starting count; indexing a missing key would raise `KeyError`. Each processed item increments exactly one count.
3. Expected $O(n)$ time and $O(m)$ extra space. For nonempty counts, `max(counts, key=counts.get)` selects a most frequent key; ties follow first-seen order. Decide an empty-result convention before using `max`.

04. Top K Frequent Elements: keep the best k

ARRAYS / FREQUENCY MAP + MIN HEAP

```
def top_k_frequent(nums, k):
    pass

print(top_k_frequent([1, 1, 1, 2, 2, 3], 2))
```

QUESTIONS

1. Return k distinct integers with the highest counts, in any order. Any tied choices are acceptable. Require $0 \leq k \leq$ the number of distinct values; otherwise raise `ValueError`.
2. Explain why a heap of size k is enough. Trace the call.
3. Give time and space costs using n inputs, m distinct values, and k. Test $k=0$ and $k=m$.

COMPACT ANSWERS

1. Python solution:

```
import heapq

def top_k_frequent(nums, k):
    counts = {}
    for value in nums:
        counts[value] = counts.get(value, 0) + 1
    if not 0 <= k <= len(counts):
        raise ValueError("k must be between 0 and distinct count")
    if k == 0:
        return []
    heap = []
    for value, count in counts.items():
        heapq.heappush(heap, (count, value))
        if len(heap) > k:
            heapq.heappop(heap)
    return [value for count, value in heap]
```

2. This code prints `[2, 1]`; the required selected values are `{1, 2}`. The heap root is the least desirable retained candidate. When size exceeds k, discard that root. Equal counts are broken by integer value in this implementation.
3. Expected $O(n + m \log(k+1))$ time and $O(m+k)$ extra space, including counts. $k=0$ returns `[]`; $k=m$ retains every distinct value. A full sort instead costs $O(n + m \log m)$.

05. Anagram detection: compare character counts

STRINGS / FREQUENCY MAP

```
def is_anagram(s, t):  
    pass  
  
print(is_anagram("anagram", "nagaram"))  
print(is_anagram("rat", "car"))
```

QUESTIONS

1. Return whether *s* and *t* contain exactly the same characters with the same counts. Comparisons are case-sensitive; spaces and punctuation count.
2. Trace the calls. Why is comparing only sets of characters insufficient?
3. Give complexity. Test empty strings, different lengths, and repeated characters.

COMPACT ANSWERS

1. Python solution:

```
def is_anagram(s, t):  
    if len(s) != len(t):  
        return False  
    counts = {}  
    for char in s:  
        counts[char] = counts.get(char, 0) + 1  
    for char in t:  
        if counts.get(char, 0) == 0:  
            return False  
        counts[char] -= 1  
    return True
```

2. Output: True, then False. A set loses multiplicity: "aab" and "abb" have the same set but are not anagrams. Every *t* character must consume one available occurrence.
3. Expected $O(n)$ time for equal-length strings and $O(u)$ space for *u* distinct characters. Both empty strings return True. The length check plus successful consumption ensures no counts remain unmatched.

06. Longest unique substring: a simple baseline

STRINGS / NESTED SCAN

```
def longest_unique_bruteforce(s):  
    pass  
  
print(longest_unique_bruteforce("abcabcbb"))  
print(longest_unique_bruteforce("bbbbbb"))
```

QUESTIONS

1. Return the length of the longest contiguous substring with no repeated characters. First solve it by trying each starting position.
2. Trace the calls. Explain why the scan for one starting position stops at its first duplicate.
3. Give complexity and explain why a subsequence is different. Improve this solution using the window in Question 21.

COMPACT ANSWERS

1. Python solution:

```
def longest_unique_bruteforce(s):  
    best = 0  
    for start in range(len(s)):  
        seen = set()  
        for end in range(start, len(s)):  
            if s[end] in seen:  
                break  
            seen.add(s[end])  
            best = max(best, end - start + 1)  
    return best
```

2. Output: 3, then 1. "abc" is valid, but extending it with the next a creates a duplicate. Any longer substring from that same start retains the duplicate, so stop.
3. Worst-case $O(n^2)$ time; $O(\min(n, u))$ extra space for u possible distinct characters. A substring is contiguous; a subsequence may skip positions. Empty input returns 0. Question 21 reuses work instead of restarting each scan.

07. String compression: read runs, write compactly

STRINGS / TWO POINTERS

```
def compress(chars):
    pass

chars = list("aabbcccccccccc")
k = compress(chars)
print(k, chars[:k])
```

QUESTIONS

1. Compress a mutable character list in place. Write each run character, followed by decimal count digits only when the run length exceeds 1. Return the valid prefix length; contents after that prefix do not matter.
2. Trace the example, including the multi-digit count 10.
3. Give complexity, test [] and one character, and explain why writing cannot overwrite unread input.

COMPACT ANSWERS

1. Python solution:

```
def compress(chars):
    read = write = 0
    while read < len(chars):
        char = chars[read]
        start = read
        while read < len(chars) and chars[read] == char:
            read += 1
        count = read - start
        chars[write] = char
        write += 1
        if count > 1:
            for digit in str(count):
                chars[write] = digit
                write += 1
    return write
```

```
7 ['a', '2', 'b', '3', 'c', '1', '0']
```

2. Runs of lengths 2, 3, 10 become a2, b3, c10. write never gets ahead of the already consumed read position because a compressed run is no longer than its original run.
3. $O(n)$ time. `str(count)` temporarily uses $O(\log n)$ space; no separate compressed list is built. `[]` returns 0 and one character returns 1. Copying `chars[:k]` in the sample is display-only extra space, not part of compress.

08. Reverse words: normalize whitespace

STRINGS / SPLIT + REVERSE

```
def reverse_words(s):  
    pass  
  
print(reverse_words("  the sky   is blue  "))
```

QUESTIONS

1. Return words in reverse order with exactly one space between them and no leading or trailing spaces. Any whitespace separates words.
2. Predict the example and an all-whitespace input. Explain `split()` versus `split(" ")`.
3. Give time and extra-space costs. Must each individual word also be reversed?

COMPACT ANSWERS

1. Python solution:

```
def reverse_words(s):  
    words = s.split()  
    words.reverse()  
    return " ".join(words)
```

2. Output: blue is sky the. All-whitespace input returns "". `split()` groups arbitrary whitespace and omits empty pieces; `split(" ")` uses literal spaces and can preserve empty pieces.
3. $O(n)$ time and $O(n)$ extra space for n input characters, including split words and the returned string. Word order changes; characters inside each word stay unchanged. Python strings are immutable, so this creates a new string.

09. Reverse a linked list: save the next node

LINKED LISTS / POINTER REWIRING

```
class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next

def reverse_list(head):
    pass

head = Node(1, Node(2, Node(3)))
head = reverse_list(head)
# Print values while following next.
```

QUESTIONS

1. Reverse a singly linked, acyclic list by changing next pointers. Return the new head; do not allocate replacement nodes.
2. Trace the list 1 -> 2 -> 3. What must be saved before changing current.next?
3. Give complexity. Test None and one node. What happens to the old head?

COMPACT ANSWERS

1. Python solution:

```
def reverse_list(head):
    previous = None
    current = head
    while current is not None:
        following = current.next
        current.next = previous
        previous = current
        current = following
    return previous
```

2. Traversal after reversal: 3, 2, 1. Save following first so the original remainder is reachable. previous heads the reversed prefix; current heads the unprocessed suffix.
3. O(n) time, O(1) extra space. None returns None. One node is unchanged. The old head becomes the tail with next=None, so the caller must use the returned head.

10. Detect a linked-list cycle: slow and fast

LINKED LISTS / TWO POINTERS

```
class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next

def has_cycle(head):
    pass

a, b, c = Node(1), Node(2), Node(3)
a.next, b.next, c.next = b, c, b
print(has_cycle(a))
```

QUESTIONS

1. Return whether following next links eventually revisits a node. Do not modify the list and use $O(1)$ extra space.
2. Explain how moving at different speeds detects the cycle. Why compare nodes with `is`?
3. Give complexity and test an acyclic list, `None`, and one node pointing to itself.

COMPACT ANSWERS

1. Python solution:

```
def has_cycle(head):
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

2. Output: `True`. Once both pointers enter a cycle, fast gains one position per step and eventually meets slow. Compare object identity; two distinct nodes may have equal values.
3. $O(n)$ time for n distinct reachable nodes, $O(1)$ space. An acyclic path eventually sends fast to `None`. `None` returns `False`; a self-loop returns `True`. Do not declare a cycle merely because both pointers start at head.

11. Find the middle node: one pass

LINKED LISTS / TWO POINTERS

```
class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next

def middle_node(head):
    pass

head = Node(1, Node(2, Node(3, Node(4))))
print(middle_node(head).val)
```

QUESTIONS

1. Return the middle node of an acyclic list. For an even number of nodes, return the second of the two middle nodes. Return None for an empty list.
2. Trace four nodes and five nodes. Why does slow move only once when fast moves twice?
3. Give complexity and describe how you would instead return the first middle for an even-length list.

COMPACT ANSWERS

1. Python solution:

```
def middle_node(head):
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
    return slow
```

2. The shown call prints 3. Five nodes also return node 3. When fast has covered the full length, slow has covered roughly half.
3. $O(n)$ time, $O(1)$ space. For the first middle, return None for empty input, then loop while fast.next is not None and fast.next.next is not None. Both versions return the single node for a one-node list.

12. Valid parentheses: match the most recent opener

STACKS / LIFO

```
def valid_parentheses(s):  
    pass  
  
print(valid_parentheses("([]){}"))  
print(valid_parentheses("([)]"))
```

QUESTIONS

1. Return whether (), [], and {} are correctly matched and nested. Reject characters other than these six bracket characters.
2. Trace both calls. Why is a stack needed instead of only counting open and closed brackets?
3. Give complexity and test empty input, a closing bracket first, and leftover open brackets.

COMPACT ANSWERS

1. Python solution:

```
def valid_parentheses(s):  
    matches = {')': '(', ']': '[', '}': '{'}  
    stack = []  
    for char in s:  
        if char in '([{':  
            stack.append(char)  
        elif char in matches:  
            if not stack or stack.pop() != matches[char]:  
                return False  
        else:  
            return False  
    return not stack
```

2. Output: True, then False. In "([)]", the closing) encounters [at the top, so nesting is invalid even though total counts match. The most recent unclosed opener must close first.
3. O(n) time and O(n) extra space. Empty input is valid. A closer with an empty stack fails immediately. Unclosed openers make the final stack nonempty.

13. Min stack: remember the minimum at every depth

STACKS / AUGMENTED STATE

```
class MinStack:
    pass # Implement push, pop, top, and get_min.

stack = MinStack()
for value in [3, 1, 1, 2]:
    stack.push(value)
print(stack.get_min())
stack.pop()
stack.pop()
print(stack.get_min())
```

QUESTIONS

1. Implement push(value), pop(), top(), and get_min(). pop returns the removed integer. Empty pop, top, and get_min raise IndexError. Every operation should have O(1) or amortized O(1) time.
2. Trace duplicate minimum values in the example.
3. Explain what each stored entry represents, and give total space for n entries.

COMPACT ANSWERS

1. Python solution:

```
class MinStack:
    def __init__(self):
        self.stack = []

    def push(self, value):
        minimum = value if not self.stack else min(value, self.get_min())
        self.stack.append((value, minimum))

    def pop(self):
        return self.stack.pop()[0]

    def top(self):
        return self.stack[-1][0]

    def get_min(self):
        return self.stack[-1][1]
```

2. Output: 1, then 1. After removing 2 and one copy of 1, the earlier minimum 1 remains available. No rescan is needed.
3. Each pair holds (value, minimum of all entries through this depth). Reading the last pair supplies the current minimum. O(n) total space; reads and pop are O(1), push is amortized O(1) because Python list append can resize. List indexing/pop naturally raises IndexError when empty.

14. Queue implementation: two stacks

QUEUES / AMORTIZED ANALYSIS

```
class MyQueue:
    pass # Implement push, pop, peek, and empty.

q = MyQueue()
q.push(10)
q.push(20)
print(q.pop())
q.push(30)
print(q.peek(), q.pop(), q.pop(), q.empty())
```

QUESTIONS

1. Implement a FIFO queue using two Python lists as stacks. pop and peek raise IndexError when empty; pop returns the oldest element.
2. Trace the calls. When should elements transfer between the stacks?
3. Give worst-case and amortized time per operation and total space. Why avoid list.pop(0) for a large queue?

COMPACT ANSWERS

1. Python solution:

```
class MyQueue:
    def __init__(self):
        self.incoming = []
        self.outgoing = []

    def push(self, value):
        self.incoming.append(value)

    def _move(self):
        if not self.outgoing:
            while self.incoming:
                self.outgoing.append(self.incoming.pop())

    def pop(self):
        self._move()
        return self.outgoing.pop()

    def peek(self):
        self._move()
        return self.outgoing[-1]

    def empty(self):
        return not self.incoming and not self.outgoing
```

```
10
20 20 30 True
```

2. Transfer only when outgoing is empty. Reversing incoming places the oldest element on top of outgoing; transferring earlier would disrupt FIFO order.
3. pop/peek can be $O(n)$ for one transfer, but amortized $O(1)$: each element transfers at most once. push is amortized $O(1)$; empty is $O(1)$. Total space $O(n)$. list.pop(0) shifts elements and costs $O(n)$. For normal application code, collections.deque is a ready-made queue.

15. Tree traversal: iterative depth-first search

TREES / EXPLICIT STACK

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def preorder(root):
    pass

root = TreeNode(1, TreeNode(2, TreeNode(4)), TreeNode(3))
print(preorder(root))
```

QUESTIONS

1. Return preorder values from a binary tree: root, left subtree, right subtree. Use a stack, without recursion.
2. Trace the example. In what order must children be pushed onto a LIFO stack?
3. Give time and extra-space costs using n nodes and tree height h . Test an empty tree.

COMPACT ANSWERS

1. Python solution:

```
def preorder(root):
    if root is None:
        return []
    result, stack = [], [root]
    while stack:
        node = stack.pop()
        result.append(node.val)
        if node.right is not None:
            stack.append(node.right)
        if node.left is not None:
            stack.append(node.left)
    return result
```

2. Output: [1, 2, 4, 3]. Push right before left, so left is popped first. A stack holds nodes still waiting to be visited.
3. $O(n)$ time, $O(h)$ auxiliary stack space ($O(n)$ upper bound), plus $O(n)$ for the returned list. Empty input returns []. Compare this explicit stack with the call stack in Question 25.

16. Tree traversal: breadth-first levels

TREES / QUEUE

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def level_order(root):
    pass

root = TreeNode(1, TreeNode(2, TreeNode(4)), TreeNode(3))
print(level_order(root))
```

QUESTIONS

1. Return a list of levels, visiting each level left to right. An empty tree returns []. Use an efficient queue.
2. Trace the example. Why take the queue length once at the start of each level?
3. Give time and auxiliary space using n nodes and maximum level width w .

COMPACT ANSWERS

1. Python solution:

```
from collections import deque

def level_order(root):
    if root is None:
        return []
    queue, levels = deque([root]), []
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)
        levels.append(level)
    return levels
```

2. Output: [[1], [2, 3], [4]]. The initial queue length fixes how many current-level nodes to remove; children appended during that loop belong to the next level. `deque.popleft` avoids shifting a list.
3. $O(n)$ time and $O(w)$ auxiliary space, plus $O(n)$ output. DFS explores a branch first; BFS visits nodes by increasing distance from the root.

17. Maximum tree depth: combine child results

TREES / RECURSION

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def max_depth(root):
    pass

root = TreeNode(1, TreeNode(2, TreeNode(4)), TreeNode(3))
print(max_depth(root))
print(max_depth(None))
```

QUESTIONS

1. Return the maximum number of nodes on a root-to-leaf path. An empty tree has depth 0; one node has depth 1.
2. Trace the calls. Identify the base case and recursive combination.
3. Give time and stack-space complexity for balanced and highly skewed trees.

COMPACT ANSWERS

1. Python solution:

```
def max_depth(root):
    if root is None:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

2. Output: 3, then 0. Each node asks both children for their depths, takes the larger one, and adds itself. Returning 0 for a missing child terminates recursion.
3. $O(n)$ time and $O(h)$ call-stack space. A balanced tree has $h=O(\log n)$; a chain has $h=O(n)$. A sufficiently deep chain can exceed Python's recursion limit; an explicit DFS stack avoids relying on that call stack. The depth convention here counts nodes, not edges.

18. Validate a BST: carry ancestor bounds

TREES / RANGE INVARIANT

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def is_valid_bst(root):
    pass

root = TreeNode(10, TreeNode(5), TreeNode(15, TreeNode(6)))
print(is_valid_bst(root))
```

QUESTIONS

1. Validate a binary search tree of integer values. Every left-subtree value must be strictly smaller, and every right-subtree value strictly larger, than each relevant ancestor. Duplicates are invalid; an empty tree is valid.
2. Explain why the example fails even though 6 is smaller than its parent 15.
3. Give time and space costs. Why is checking only immediate children insufficient?

COMPACT ANSWERS

1. Python solution:

```
def is_valid_bst(root):
    stack = [(root, float('-inf'), float('inf'))]
    while stack:
        node, low, high = stack.pop()
        if node is None:
            continue
        if not low < node.val < high:
            return False
        stack.append((node.right, node.val, high))
        stack.append((node.left, low, node.val))
    return True
```

2. Output: False. Node 6 belongs to 10's right subtree, so it must exceed 10. Its inherited range is (10, 15), which excludes 6.
3. $O(n)$ time, $O(h)$ auxiliary stack space ($O(n)$ upper bound). Each stored range carries all ancestor restrictions. Strict comparisons reject duplicates. Immediate-parent checks miss violations higher in the tree.

19. Maximum sum subarray: restart or extend

ARRAY SCANNING / KADANE'S ALGORITHM

```
def max_subarray(nums):  
    pass  
  
print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))  
print(max_subarray([-5, -2, -7]))
```

QUESTIONS

1. Return the largest sum of a nonempty contiguous subarray. Integers may be positive, zero, or negative. Reject an empty input with `ValueError`.
2. Trace the calls. At each position, choose between extending the previous subarray and starting at the current value.
3. Give complexity. Is this the same problem as a fixed-size sliding window? Why not initialize `best` to 0?

COMPACT ANSWERS

1. Python solution:

```
def max_subarray(nums):  
    if not nums:  
        raise ValueError("nums must be nonempty")  
    current = best = nums[0]  
    for i in range(1, len(nums)):  
        current = max(nums[i], current + nums[i])  
        best = max(best, current)  
    return best
```

2. Output: 6, then -2. The first best segment is [4, -1, 2, 1]. `current` is the best nonempty sum ending at this index; `best` is the best sum seen anywhere.
3. $O(n)$ time and $O(1)$ extra space. The length is unrestricted, so use Kadane's recurrence rather than a fixed window. Starting at 0 would incorrectly allow an empty subarray to beat every negative value.

20. Fixed-size window: maximum sum of k items

SLIDING WINDOW / ADD ONE, REMOVE ONE

```
def max_window_sum(nums, k):
    pass

print(max_window_sum([2, 1, 5, 1, 3, 2], 3))
print(max_window_sum([-4, -2, -7], 2))
```

QUESTIONS

1. Return the largest sum of exactly k contiguous values. Require $1 \leq k \leq \text{len}(\text{nums})$, otherwise raise `ValueError`. Negative values are allowed.
2. Trace the calls. How do you update a window without summing its k elements again?
3. Give complexity. Test $k=1$ and $k=\text{len}(\text{nums})$, and compare with Question 19.

COMPACT ANSWERS

1. Python solution:

```
def max_window_sum(nums, k):
    if not 1 <= k <= len(nums):
        raise ValueError("k must be between 1 and len(nums)")
    total = sum(nums[i] for i in range(k))
    best = total
    for right in range(k, len(nums)):
        total += nums[right] - nums[right - k]
        best = max(best, total)
    return best
```

2. Output: 9, then -6. First-example window sums are 8, 7, 9, 6. Add the entering element and subtract the element that just left.
3. $O(n)$ time, $O(1)$ extra space. The initial sum uses a generator rather than a copied slice. $k=1$ selects the largest element; $k=n$ returns the full sum. Here the length is fixed; in Question 19 it can vary.

21. Longest unique substring: reuse a moving window

SLIDING WINDOW / LAST-SEEN INDEX

```
def longest_unique(s):
    pass

print(longest_unique("abcabcbb"))
print(longest_unique("abba"))
```

QUESTIONS

1. Solve Question 6 in expected $O(n)$ time. Return the longest contiguous substring length with no repeated characters.
2. Trace "abba". Why must the left boundary never move backward?
3. Give time and space costs. State the invariant maintained after each character.

COMPACT ANSWERS

1. Python solution:

```
def longest_unique(s):
    last_seen = {}
    left = best = 0
    for right, char in enumerate(s):
        left = max(left, last_seen.get(char, -1) + 1)
        last_seen[char] = right
        best = max(best, right - left + 1)
    return best
```

2. Output: 3, then 2. Trace for "abba":

right / char	left after update	window	best
0 / a	0	a	1
1 / b	0	ab	2
2 / b	2	b	2
3 / a	2	ba	2

3. Expected $O(n)$ time, $O(u)$ extra space for u distinct characters encountered. $s[\text{left}:\text{right}+1]$ contains no repeats. `max` preserves `left` when a previous occurrence is already outside the window; without it, the final `a` in "abba" would move `left` backward and include the duplicate `b`.

22. Minimum-length subarray reaching a target

SLIDING WINDOW / POSITIVE VALUES

```
def min_subarray_len(nums, target):
    pass

print(min_subarray_len([2, 3, 1, 2, 4, 3], 7))
print(min_subarray_len([1, 1, 1], 5))
```

QUESTIONS

1. Given positive integers, return the shortest contiguous subarray length whose sum is at least a positive target. Return 0 when none exists. Reject target ≤ 0 with ValueError.
2. Trace the calls. When should the left boundary advance?
3. Give complexity. Explain why this shrinking rule relies on the positive-input assumption.

COMPACT ANSWERS

1. Python solution:

```
def min_subarray_len(nums, target):
    if target <= 0:
        raise ValueError("target must be positive")
    left = total = 0
    best = len(nums) + 1
    for right, value in enumerate(nums):
        total += value
        while total >= target:
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best > len(nums) else best
```

2. Output: 2, then 0. [4, 3] reaches 7 using two values. Once a window qualifies, repeatedly remove its leftmost value to look for a shorter qualifying window.
3. $O(n)$ time, $O(1)$ extra space: each boundary advances at most n times. Positive values make expansion increase and removal decrease the sum. Negative inputs break that monotonic reasoning and require a different algorithm. Empty input returns 0.

23. Longest substring with at most k distinct characters

SLIDING WINDOW / COUNT MAP

```
def longest_at_most_k(s, k):
    pass

print(longest_at_most_k("eceba", 2))
print(longest_at_most_k("aa", 0))
```

QUESTIONS

1. Return the longest contiguous substring length containing at most k distinct characters. Require $k \geq 0$, otherwise raise `ValueError`.
2. Trace the calls. Why delete a character from the count map when its count becomes zero?
3. Give complexity. Compare this window condition with the no-repeated-character condition in Question 21.

COMPACT ANSWERS

1. Python solution:

```
def longest_at_most_k(s, k):
    if k < 0:
        raise ValueError("k must be nonnegative")
    counts = {}
    left = best = 0
    for right, char in enumerate(s):
        counts[char] = counts.get(char, 0) + 1
        while len(counts) > k:
            outgoing = s[left]
            counts[outgoing] -= 1
            if counts[outgoing] == 0:
                del counts[outgoing]
            left += 1
        best = max(best, right - left + 1)
    return best
```

2. Output: 3, then 0. "ece" has two distinct characters despite repeating e. Zero-count entries must disappear so `len(counts)` measures only characters currently in the window.
3. Expected $O(n)$ time and $O(\min(n, k+1))$ auxiliary space. Each character enters and leaves at most once. Repeated occurrences are allowed here, unlike Question 21; only the number of distinct characters is bounded. Empty input or $k=0$ gives 0.

24. Fibonacci: cache repeated recursive work

RECURSION / MEMOIZATION

```
def fibonacci(n):
    pass

print(fibonacci(0), fibonacci(1), fibonacci(6))
```

QUESTIONS

1. Return $F(n)$, with $F(0)=0$, $F(1)=1$, and $F(n)=F(n-1)+F(n-2)$. Require a nonnegative integer n . Raise `ValueError` for a negative n .
2. Implement recursion with a cache. Explain why naive recursion recomputes the same values.
3. Give time and space costs. How can iteration reduce auxiliary space?

COMPACT ANSWERS

1. Python solution:

```
def fibonacci(n):
    if n < 0:
        raise ValueError("n must be nonnegative")
    cache = {0: 0, 1: 1}

    def solve(i):
        if i not in cache:
            cache[i] = solve(i - 1) + solve(i - 2)
        return cache[i]

    return solve(n)
```

2. Output: 0 1 8. Computing $F(6)$ needs $F(5)$ and $F(4)$; computing $F(5)$ also needs $F(4)$. The cache lets each distinct state be evaluated only once.
3. $O(n)$ additions, $O(n)$ cache and call-stack space. Naive recursion is exponential ($O(2^n)$ upper bound). Iteration can keep only two previous values, using $O(1)$ scalar storage. Python recursion has a depth limit; large n favors iteration.

25. Recursive tree traversal: base case and visit order

RECURSION / DEPTH-FIRST SEARCH

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def inorder(root):
    pass

root = TreeNode(4, TreeNode(2, TreeNode(1), TreeNode(3)),
                TreeNode(6, TreeNode(5), TreeNode(7)))
print(inorder(root))
```

QUESTIONS

1. Return inorder values: left subtree, node, right subtree. Use recursion and a shared result list.
2. Trace the example. How would the visit order change for preorder or postorder?
3. Give complexity using n nodes and height h . Why is `[]` the output for an empty tree?

COMPACT ANSWERS

1. Python solution:

```
def inorder(root):
    result = []

    def visit(node):
        if node is None:
            return
        visit(node.left)
        result.append(node.val)
        visit(node.right)

    visit(root)
    return result
```

2. Output: `[1, 2, 3, 4, 5, 6, 7]`. Preorder appends before visiting either child. Postorder appends after visiting both. The call stack remembers where to resume after each child returns.
3. $O(n)$ time, $O(h)$ call-stack space, plus $O(n)$ output. The base case returns without appending, so `None` gives `[]`. This particular tree is a BST, so inorder is sorted; inorder of an arbitrary binary tree need not be sorted.

26. Backtracking basics: enumerate subsets

RECURSION / CHOOSE, EXPLORE, UNDO

```
def subsets(nums):
    pass

print(subsets([1, 2]))
print(len(subsets([1, 2, 3])))
```

QUESTIONS

1. Return every subset of a list of distinct integers, including the empty subset. Any ordering is acceptable. Do not modify nums.
2. Explain the choose/explore/undo steps. Why append `path.copy()` rather than `path` itself?
3. Give time and space costs, counting output separately. Test an empty input.

COMPACT ANSWERS

1. Python solution:

```
def subsets(nums):
    result, path = [], []

    def visit(start):
        result.append(path.copy())
        for i in range(start, len(nums)):
            path.append(nums[i])
            visit(i + 1)
            path.pop()

    visit(0)
    return result
```

```
[[[]], [1], [1, 2], [2]]
8
```

2. Append a choice, recurse using only later positions, then pop to restore the earlier path. A copy freezes the current subset; saving the same mutable path would let later changes alter every saved reference.
3. $O(n \cdot 2^n)$ time including copying; $O(n)$ auxiliary path/call-stack space and $O(n \cdot 2^n)$ output space. There are 2^n subsets. Empty input returns `[[[]]]`. Distinct input values avoid duplicate subsets.

27. Backtracking basics: enumerate permutations

RECURSION / USED-CHOICE TRACKING

```
def permutations(nums):
    pass

print(permutations([1, 2, 3]))
```

QUESTIONS

1. Return all orderings of a list of distinct integers. Each value must be used exactly once in each ordering. Any output ordering is acceptable.
2. Explain how a used array differs from the increasing start index in subset generation.
3. Give complexity including output. Predict the result for an empty list.

COMPACT ANSWERS

1. Python solution:

```
def permutations(nums):
    result, path = [], []
    used = [False] * len(nums)

    def visit():
        if len(path) == len(nums):
            result.append(path.copy())
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            visit()
            path.pop()
            used[i] = False

    visit()
    return result
```

2. Output order: [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1], inside an outer list. Every unused position remains eligible at every depth; subsets instead only move forward to avoid reordering the same selection.
3. $O(n \cdot n!)$ time including output copies; $O(n)$ auxiliary state/stack space and $O(n \cdot n!)$ output space. Empty input returns `[[[]]]`, representing one empty ordering. Restore both path and used after each recursive branch.

28. Binary search: shrink a sorted search interval

EXTRA FUNDAMENTAL / BINARY SEARCH

```
def binary_search(nums, target):  
    pass  
  
print(binary_search([1, 3, 5, 7], 5))  
print(binary_search([1, 3, 5, 7], 4))
```

QUESTIONS

1. Return an index of target in a nondecreasing sorted list, or -1 if absent. Any matching index is acceptable when duplicates exist.
2. Trace both calls. State whether your interval endpoints are inclusive.
3. Give complexity and explain why $\text{left} = \text{mid}$ or $\text{right} = \text{mid}$ can cause an infinite loop.

COMPACT ANSWERS

1. Python solution:

```
def binary_search(nums, target):  
    left, right = 0, len(nums) - 1  
    while left <= right:  
        mid = (left + right) // 2  
        if nums[mid] == target:  
            return mid  
        if nums[mid] < target:  
            left = mid + 1  
        else:  
            right = mid - 1  
    return -1
```

2. Output: 2, then -1. The candidate interval is inclusive $[\text{left}, \text{right}]$. Comparing with the midpoint discards a half that cannot contain target.
3. $O(\log n)$ time, $O(1)$ extra space. Moving past mid guarantees progress after ruling it out. Empty input begins with $\text{right} = -1$ and returns -1. The sorted-input assumption is essential.

29. Sorted Two Sum: move inward from both ends

EXTRA FUNDAMENTAL / TWO POINTERS

```
def two_sum_sorted(nums, target):  
    pass  
  
print(two_sum_sorted([1, 2, 4, 7], 6))  
print(two_sum_sorted([3, 3], 6))
```

QUESTIONS

1. For a nondecreasing sorted integer list, return two distinct zero-based indices summing to target. Return [] if none exists; any valid pair is acceptable.
2. Trace the calls. Explain which pointer moves when the sum is too small or too large.
3. Give complexity and compare with the unsorted Two Sum solution in Question 1.

COMPACT ANSWERS

1. Python solution:

```
def two_sum_sorted(nums, target):  
    left, right = 0, len(nums) - 1  
    while left < right:  
        total = nums[left] + nums[right]  
        if total == target:  
            return [left, right]  
        if total < target:  
            left += 1  
        else:  
            right -= 1  
    return []
```

2. Output: [1, 2], then [0, 1]. Too small: increase left to get a larger value. Too large: decrease right to get a smaller value. $left < right$ prevents reusing one index.
3. $O(n)$ time, $O(1)$ space. Sorted order makes pointer movement safe. The unsorted hash-map approach uses expected $O(n)$ time and $O(n)$ space; sorting an unsorted list would add sorting cost and require preserving original indices if those are requested.

30. Climbing stairs: reuse two smaller answers

EXTRA FUNDAMENTAL / DYNAMIC PROGRAMMING

```
def climb_stairs(n):
    pass

print(climb_stairs(0), climb_stairs(1), climb_stairs(4))
```

QUESTIONS

1. Count ordered ways to climb n steps using moves of 1 or 2 steps. Let $n=0$ have one empty sequence; require nonnegative integer n , raising `ValueError` for a negative n .
2. Derive the recurrence by considering the last move. Use iteration with two stored values.
3. Give complexity and explain its relationship to Fibonacci.

COMPACT ANSWERS

1. Python solution:

```
def climb_stairs(n):
    if n < 0:
        raise ValueError("n must be nonnegative")
    previous, current = 1, 1
    for _ in range(n):
        previous, current = current, previous + current
    return previous
```

2. Output: 1 1 5. A sequence ending in a 1-step move follows a solution for $n-1$; one ending in a 2-step move follows a solution for $n-2$. Thus $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$, with $\text{ways}(0) = \text{ways}(1) = 1$.
3. $O(n)$ additions and $O(1)$ scalar auxiliary storage. This is $F(n+1)$ under Question 24's Fibonacci definition. Python can represent the growing integer exactly; actual bit storage and arithmetic cost grow for very large n .